

Centroid Decomposition

Sean

October 4, 2026



Outline

- Finding a centroid
- Building the decomposition
- Counting paths through a centroid
- C++ implementation and complexity



Example: Fixed-Length Paths I

CSES: Fixed-Length Paths I

Count the distinct paths in an unweighted tree that contain exactly k edges.

- $1 \leq k \leq n \leq 2 \cdot 10^5$
- A path and its reversal count as the same path
- Equivalently, count unordered pairs of distinct vertices at distance k



A balanced split of a tree

Trying every pair of endpoints takes $O(n^2)$ work.

Divide and conquer

Choose a vertex, handle paths that go through it, then solve the remaining components independently.

- A balanced split makes every new component much smaller
- A **centroid** guarantees that balance



- 1 Finding a centroid
- 2 Building the decomposition
- 3 Counting paths through a centroid
- 4 Implementation and complexity

Finding a centroid

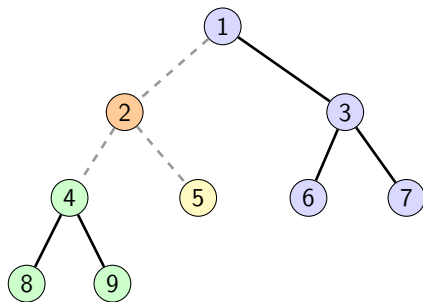
What is a centroid?

In a tree with m vertices, a **centroid** is a vertex whose removal leaves no component larger than $\lfloor m/2 \rfloor$.

- Balance counts vertices, not distances or edge weights
- The definition does not depend on an initial root
- If two centroids exist, either one works
- A single vertex is its own centroid



Removing vertex 2



Dashed edges are cut. Vertex 2 is a centroid.

Here $m = 9$, so every remaining piece must have size at most 4.

- $\{1, 3, 6, 7\}$: size 4
- $\{4, 8, 9\}$: size 3
- $\{5\}$: size 1

Checking the size of every piece

Temporarily root the current component and compute subtree sizes.

- Removing u leaves one piece for each child v , of size $\text{sz}[v]$
- The piece toward its parent has size $m - \text{sz}[u]$

$$\text{largest}[u] = \max \left(m - \text{sz}[u], \max_{v \text{ child of } u} \text{sz}[v] \right)$$

A missing parent-side piece has size 0. If there are no children, use only the parent-side size.



Testing candidates in the example

Root the nine-vertex tree at 1 just to compute sizes.

Candidate	Parent side	Child sides	Largest piece
1	0	5, 3	5
2	4	3, 1	4
3	6	1, 1	6

Balance test

A vertex is a centroid exactly when $\text{largest}[u] \leq \lfloor m/2 \rfloor$.



Why a centroid always exists

- If the largest piece has size $s > m/2$, move to the neighboring vertex inside it
- The piece back toward the old vertex now has size $m - s < m/2$
- Every other piece has fewer than s vertices

A decreasing quantity

The largest-piece size strictly decreases on every move. Eventually it is at most $\lfloor m/2 \rfloor$.



Practice: Finding a Centroid

CSES: Finding a Centroid

Output any centroid of the given tree.

- One traversal computes parents and subtree sizes
- Check each vertex's largest remaining piece
- On the full tree, this takes $O(n)$ time

This problem needs one centroid search. The next step applies it repeatedly to smaller components.



- 1 Finding a centroid
- 2 Building the decomposition
- 3 Counting paths through a centroid
- 4 Implementation and complexity

Building the decomposition

Repeating the balanced split

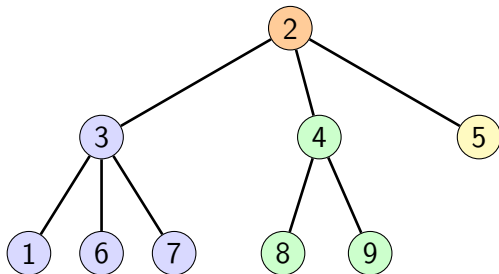
For each active connected component:

- 1 Find a centroid c using this component's size m
- 2 Mark c removed to block later traversals
- 3 Process the paths that go through c
- 4 Decompose each remaining component

Marking c only blocks traversal; it can still be an endpoint. Recompute subtree sizes inside every new component.



The centroid tree



Each child represents the next centroid chosen in one remaining component.

These edges describe the decomposition. For example, 2–3 is not an edge of the original tree.

Only logarithmically many levels

Each split at least halves the component size:

$$n, \quad \leq \frac{n}{2}, \quad \leq \frac{n}{4}, \quad \leq \frac{n}{8}, \quad \dots$$

- Every branch has $O(\log n)$ levels
- Each vertex is selected as a centroid exactly once

What still needs care

$O(n)$ work per level gives $O(n \log n)$ in total.



Which level counts a path?

At a centroid c , a path has one of two forms:

- It passes through c : handle it at this level
- It stays inside one remaining component: defer it to that component

Exactly one counting level

Follow the components containing both endpoints. Count the path at the first centroid that separates the endpoints or is one of them.

Later components cannot contain both endpoints, so the path cannot be counted again.



- 1 Finding a centroid
- 2 Building the decomposition
- 3 Counting paths through a centroid**
- 4 Implementation and complexity

Counting paths through a centroid

Two distances from the centroid

If a and b lie in different components after removing c , their path goes through c .

$$\text{dist}(a, b) = \text{dist}(a, c) + \text{dist}(c, b)$$

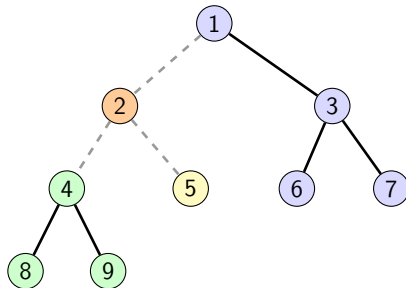
A path of exactly k edges

A vertex at depth d from c needs a partner at depth $k - d$.

The centroid itself is a possible endpoint at depth 0.



Depths in each neighbor component



For $c = 2$ and $k = 2$:

- Through 1: depths $[1, 2]$
- Through 4: depths $[1, 2, 2]$
- Through 5: depths $[1]$

Vertices 6, 7 have depth 3, so skip them at this centroid. They remain available in later components.

A frequency table of earlier components

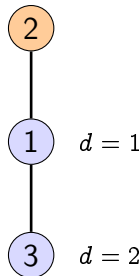
Before processing one neighbor component, let $\text{freq}[d]$ count vertices at depth d among:

- The centroid itself
- Previously processed neighbor components only

- 1 Start with $\text{freq}[0] = 1$
- 2 For each collected depth $d \leq k$ in the current component, add $\text{freq}[k - d]$ to the answer
- 3 After all queries for this component, insert all its depths



Why the current component must wait



Suppose $k = 3$ at centroid 2.

- Vertices 1 and 3 have depth sum $1 + 2 = 3$
- Their actual distance is 1, because their path stays in one component
- Mixing queries with insertions could count this pair incorrectly

Query the entire component first. Insert its depths afterward.

Counting at centroid 2

Return to $k = 2$. Process the components through 1, then 4, then 5.

Component	Depths	New pairs	$[f_0, f_1, f_2]$ after
Initial			$[1, 0, 0]$
Through 1	$[1, 2]$	1	$[1, 1, 1]$
Through 4	$[1, 2, 2]$	3	$[1, 2, 3]$
Through 5	$[1]$	2	$[1, 3, 3]$

Paths counted at 2 : $1 + 3 + 2 = 6$

Each pair is counted when its later component is queried. Here $f_d = \text{freq}[d]$.



Finishing the smaller components

Centroid	New length-2 paths	Count
2	Counted in the previous trace	6
3	(1, 6), (1, 7), (6, 7)	3
4	(8, 9)	1
Others	Single-vertex components	0

$$\text{Total} = 6 + 3 + 1 = 10$$

For example, the path 8–4–9 stayed in one component at centroid 2, then was counted at centroid 4.



Resetting the frequency table

- Allocate `freq` once, with indices 0 through k
- Record each depth whose frequency becomes nonzero
- After processing the centroid, reset only those entries

Why this matters

Clearing all $k + 1$ entries after every centroid costs $O(nk)$, which can be quadratic.

The number of touched depths is at most the number of collected vertices plus the centroid.



- 1 Finding a centroid
- 2 Building the decomposition
- 3 Counting paths through a centroid
- 4 Implementation and complexity

Implementation and complexity

C++ setup and conventions

- C++17, standard headers, and using namespace std;
- Read n , k and adjacency list g for vertices 1 to n
- Put the following snippets in one scope after reading the tree

```
vector<char> removed(n + 1, false);  
vector<int> parent(n + 1), sz(n + 1);  
vector<int> centroidParent(n + 1, 0);  
vector<int> freq(k + 1, 0);
```

Vertex 0 means no parent. This solution assumes $1 \leq k \leq n$.



C++: collect the active component

```
auto findCentroid = [&](int entry) {  
    vector<int> order{entry};  
    parent[entry] = 0;  
    for (int i = 0; i < (int)order.size(); ++i) {  
        int u = order[i];  
        sz[u] = 1;  
        for (int v : g[u]) {  
            if (removed[v] || v == parent[u]) continue;  
            parent[v] = u;  
            order.push_back(v);  
        }  
    }  
}
```

The helper continues on the next two slides. Removed vertices block traversal.



C++: sizes within this component

Continuation of findCentroid.

```
int m = (int)order.size();
for (int i = m - 1; i > 0; --i) {
    int u = order[i];
    sz[parent[u]] += sz[u];
}
```

- Each child's size is ready before it reaches its parent
- Only vertices in this active component contribute
- The balance test uses m , not the original n



C++: the balance test

Final part of findCentroid.

```
for (int u : order) {  
    int largest = m - sz[u];  
    for (int v : g[u])  
        if (!removed[v] && parent[v] == u)  
            largest = max(largest, sz[v]);  
    if (largest <= m / 2) return u;  
}  
return -1; // A nonempty tree always has a centroid.  
};
```

The first valid candidate is enough. Any choice among tied centroids is valid.



C++: collect depths without recursion

```
auto collectDepths = [&](int start, int p) {  
    vector<int> depths;  
    vector<array<int, 3>> todo{{start, p, 1}};  
    while (!todo.empty()) {  
        auto [u, par, d] = todo.back(); todo.pop_back();  
        if (d > k) continue;  
        depths.push_back(d);  
        for (int v : g[u])  
            if (!removed[v] && v != par)  
                todo.push_back({v, u, d + 1});  
    }  
    return depths;  
};
```



C++: query earlier components

Call this helper after marking the centroid removed.

```
auto countThrough = [&](int c) {  
    long long count = 0;  
    vector<int> touched{0};  
    freq[0] = 1;  
    for (int v : g[c]) if (!removed[v]) {  
        auto depths = collectDepths(v, c);  
        for (int d : depths) count += freq[k - d];  
    }  
}
```

All vertices in this component query the same table of earlier components. Insertion follows on the next slide.



C++: insert depths and clear the table

Continuation of the same helper, inside the neighbor loop.

```
    for (int d : depths) {  
        if (freq[d] == 0) touched.push_back(d);  
        ++freq[d];  
    }  
}  
for (int d : touched) freq[d] = 0;  
return count;  
};
```

Each touched depth is recorded once. The table is empty for the next centroid.



C++: the decomposition loop

```
long long answer = 0;
vector<pair<int, int>> tasks{{1, 0}};
while (!tasks.empty()) {
    auto [entry, p] = tasks.back(); tasks.pop_back();
    int c = findCentroid(entry);
    centroidParent[c] = p;
    removed[c] = true;
    answer += countThrough(c);
    for (int v : g[c])
        if (!removed[v]) tasks.push_back({v, c});
}
cout << answer << '\n';
```

centroidParent stores the decomposition tree. Its root has parent 0.



Why the implementation is efficient

- At each decomposition level, active components are disjoint
- Every active vertex's adjacency list is scanned only a constant number of times at that level
- Even with removed neighbors retained, the sum of original degrees is at most $2(n - 1)$
- Counting and clearing touched depths also take $O(n)$ work per level

Total time

$O(n)$ work per level and $O(\log n)$ levels give $O(n \log n)$.



Memory, integers, and stack depth

- Graph, work arrays, and pending tasks use $O(n)$ memory
- The frequency table uses $O(k)$ memory, and $k \leq n$
- The answer may approach $n(n - 1)/2$, so use `long long`

Two different depths

The decomposition has $O(\log n)$ levels, but an ordinary DFS inside one component can be $O(n)$ deep. All our traversals are iterative.



Common mistakes

- Using the full tree's n in a smaller component's balance test
- Forgetting the parent-side piece of size $m - \text{sz}[u]$
- Reusing subtree sizes from before earlier centroids were removed
- Inserting part of a component before finishing its queries
- Clearing the entire frequency array after every centroid



Practice roadmap

- CSES: Finding a Centroid

Verify the balance condition, including the parent side

- CSES: Fixed-Length Paths I

Count paths once using depth frequencies and decomposition

Useful small checks

A single vertex, a chain, a star, and k larger than the tree's diameter.



The centroid decomposition workflow

- 1 Recompute sizes inside the current component
- 2 Choose a vertex whose removal leaves only small pieces
- 3 Count the paths through that centroid
- 4 Repeat inside the remaining components

For exact-length path counting

Query a component's depths before inserting them. This separates local counting from the remaining subproblems.

The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

